

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 7
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
О.А. Козюта
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

- 1) Изложение цели работы.
- 2) Задание по лабораторной работе с описанием своего варианта.
- 3) Спецификации ввода-вывода программы.
- 4) Текст программы (кратко).
- 5) Выводы по проделанной работе.

Задание 1. Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
 - Организовать ввод параметров для создания объектов (использовать экранные компоненты);
 - Осуществить визуализацию графических примитивов
- 8) Задать движение по экрану строк (одна за другой) из списка строк. Направление движения по форме и значение каждой строки выбираются случайным образом.

Задание 2. Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

8) Кривая дракона

Код:

Task1.py

"""

Moving text strings across the screen.

Each string moves in random direction; speed and pause can be controlled.

"""

```
import random
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
from PIL import ImageGrab # for screenshots
import os
```

```
class MovingTextApp:
```

```
    """Main application for moving text strings."""
```

```
    def __init__(self, root):
        self.root = root
        self.root.title("Moving Text Strings")
        self.root.geometry("800x600")
```

```
    # Parameters
```

```
    self.texts = ["Hello", "World", "Python", "Tkinter", "Animation",
```

```

        "Dragons", "Random", "Speed", "Pause", "Screenshot"]
self.speed = 10 # milliseconds per frame
self.running = True
self.strings = [] # list of moving string objects

# Canvas
self.canvas = tk.Canvas(root, bg='white', width=800, height=500)
self.canvas.pack(fill=tk.BOTH, expand=True)

# Control frame
control_frame = ttk.Frame(root)
control_frame.pack(fill=tk.X, padx=5, pady=5)

    ttk.Button(control_frame, text="Start", command=self.start_animation).pack(side=tk.LEFT,
padx=5)
    ttk.Button(control_frame, text="Pause",
command=self.pause_animation).pack(side=tk.LEFT, padx=5)
    ttk.Button(control_frame, text="Screenshot",
command=self.take_screenshot).pack(side=tk.LEFT, padx=5)
    ttk.Button(control_frame, text="Add Random String",
command=self.add_random_string).pack(side=tk.LEFT, padx=5)

    ttk.Label(control_frame, text="Speed (ms):").pack(side=tk.LEFT, padx=(20, 5))
    self.speed_var = tk.IntVar(value=self.speed)
    speed_spin = ttk.Spinbox(control_frame, from_=1, to=100, textvariable=self.speed_var,
width=5, command=self.update_speed)
    speed_spin.pack(side=tk.LEFT)

# Create initial strings
self.create_initial_strings()

# Start animation
self.animate()

def create_initial_strings(self):
    """Create several random moving strings."""
    for _ in range(5):
        self.add_random_string()

def add_random_string(self):
    """Add a new moving string with random position, direction, and text."""
    text = random.choice(self.texts)
    x = random.randint(50, self.canvas.winfo_width() - 50)
    y = random.randint(50, self.canvas.winfo_height() - 50)
    # random direction vector (dx, dy) from -3 to 3, not both zero
    dx = random.choice([-2, -1, 1, 2])
    dy = random.choice([-2, -1, 0, 1, 2])
    if dx == 0 and dy == 0:
        dy = 1
    color = random.choice(['red', 'blue', 'green', 'purple', 'orange', 'black'])

```

```

font = ('Arial', random.randint(12, 24))
item_id = self.canvas.create_text(x, y, text=text, fill=color, font=font)
self.strings.append({
    'id': item_id,
    'dx': dx,
    'dy': dy,
    'text': text,
    'color': color,
    'font': font
})

def update_speed(self):
    """Update animation speed from spinbox."""
    self.speed = self.speed_var.get()

def start_animation(self):
    """Resume animation."""
    self.running = True

def pause_animation(self):
    """Pause animation."""
    self.running = False

def take_screenshot(self):
    """Save current canvas content as PNG file."""
    try:
        # Get canvas bounding box
        x = self.root.winfo_rootx() + self.canvas.winfo_x()
        y = self.root.winfo_rooty() + self.canvas.winfo_y()
        x1 = x + self.canvas.winfo_width()
        y1 = y + self.canvas.winfo_height()
        img = ImageGrab.grab(bbox=(x, y, x1, y1))
        # Create unique filename
        counter = 1
        while os.path.exists(f"screenshot_{counter}.png"):
            counter += 1
        filename = f"screenshot_{counter}.png"
        img.save(filename)
        messagebox.showinfo("Screenshot", f"Saved as {filename}")
    except Exception as e:
        messagebox.showerror("Error", f"Could not take screenshot: {e}")

def animate(self):
    """Move all strings and update canvas."""
    if self.running:
        for obj in self.strings:
            # Move
            self.canvas.move(obj['id'], obj['dx'], obj['dy'])
            # Get current position
            pos = self.canvas.coords(obj['id'])

```

```

        if pos:
            x, y = pos[0], pos[1]
            # Bounce off walls
            if x < 10 or x > self.canvas.winfo_width() - 10:
                obj['dx'] = -obj['dx']
            if y < 10 or y > self.canvas.winfo_height() - 10:
                obj['dy'] = -obj['dy']
            # Schedule next frame
            self.root.after(self.speed, self.animate)

if __name__ == "__main__":
    root = tk.Tk()
    app = MovingTextApp(root)
    root.mainloop()

task2.py
"""
Dragon Curve fractal drawing with adjustable recursion depth.
"""

import tkinter as tk
from tkinter import ttk, messagebox
import math

class DragonCurveApp:
    """Draw Dragon Curve fractal on Canvas."""

    def __init__(self, root):
        self.root = root
        self.root.title("Dragon Curve Fractal")
        self.root.geometry("900x700")

        # Parameters
        self.depth = 10
        self.scale = 5
        self.angle = 90 # turning angle in degrees

        # Canvas
        self.canvas = tk.Canvas(root, bg='white', width=800, height=600)
        self.canvas.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Control frame
        control_frame = ttk.Frame(root)
        control_frame.pack(fill=tk.X, padx=10, pady=5)

        ttk.Label(control_frame, text="Depth (1-18):").pack(side=tk.LEFT, padx=5)
        self.depth_var = tk.IntVar(value=self.depth)
        depth_spin = ttk.Spinbox(control_frame, from_=1, to=18, textvariable=self.depth_var,

```

```

        width=5, command=self.redraw)
depth_spin.pack(side=tk.LEFT, padx=5)

    ttk.Label(control_frame, text="Scale (pixels per step):").pack(side=tk.LEFT, padx=5)
    self.scale_var = tk.IntVar(value=self.scale)
    scale_spin = ttk.Spinbox(control_frame, from_=2, to=20, textvariable=self.scale_var,
                             width=5, command=self.redraw)
    scale_spin.pack(side=tk.LEFT, padx=5)

    ttk.Button(control_frame, text="Redraw", command=self.redraw).pack(side=tk.LEFT,
padx=10)
    ttk.Button(control_frame, text="Save as EPS",
command=self.save_eps).pack(side=tk.LEFT, padx=5)

    self.redraw()

def generate_dragon_curve(self, depth):
    """
    Generate dragon curve instructions as a string of turns.
    Using the classic algorithm:
    Start with "R" (right turn). For each next iteration:
    new = old + "R" + reverse(flip(old))
    where flip changes L<->R.
    """
    if depth == 0:
        return ""
    seq = "R"
    for _ in range(depth - 1):
        flipped = seq[::-1].replace('L', 'x').replace('R', 'L').replace('x', 'R')
        seq = seq + "R" + flipped
    return seq

def draw_curve(self, seq):
    """Draw the dragon curve on canvas given the turn sequence."""
    self.canvas.delete("all")
    if not seq:
        return

    scale = self.scale_var.get()
    x, y = self.canvas.winfo_width() // 2, self.canvas.winfo_height() // 2
    angle = 0 # current direction angle in degrees (0 = right)
    points = [(x, y)]

    for turn in seq:
        if turn == 'R':
            angle += self.angle
        else: # 'L'
            angle -= self.angle
        # Move forward
        x += scale * math.cos(math.radians(angle))

```

```

        y += scale * math.sin(math.radians(angle))
        points.append((x, y))

# Draw lines
for i in range(len(points) - 1):
    self.canvas.create_line(points[i][0], points[i][1],
                            points[i+1][0], points[i+1][1],
                            fill='darkblue', width=2)

# Optionally adjust scroll region
self.canvas.config(scrollregion=self.canvas.bbox("all"))

def redraw(self):
    """Regenerate fractal with current depth and scale."""
    depth = self.depth_var.get()
    if depth < 1:
        depth = 1
    if depth > 18:
        depth = 18
    self.depth_var.set(depth)
    seq = self.generate_dragon_curve(depth)
    self.draw_curve(seq)

def save_eps(self):
    """Save canvas content as EPS file."""
    try:
        self.canvas.postscript(file="dragon_curve.eps", colormode='color')
        messagebox.showinfo("Save", "Saved as dragon_curve.eps")
    except Exception as e:
        messagebox.showerror("Error", f"Could not save EPS: {e}")

if __name__ == "__main__":
    root = tk.Tk()
    app = DragonCurveApp(root)
    root.mainloop()

```

Вывод: освоены возможности языка программирования Python в разработке оконных приложений.